

Application: Speech Recognition Processing System

Aim

To analyze the recurrence relation $T(n) = 2T(n/2) + \sqrt{n}$ using the Master Theorem and determine the time complexity of a speech recognition processing system.

Problem Statement

A speech recognition system uses a divide-and-conquer algorithm to process audio data. The recurrence relation is:

$$T(n) = 2T(n/2) + \sqrt{n}$$

Apply the Master Theorem to determine the time complexity by identifying the parameters, selecting the correct case, and analyzing the algorithm's efficiency for large audio inputs.

Algorithm

Step 1: Start the program.

Step 2: Read the input size n .

Step 3: Set

- $a = 2$
- $b = 2$
- $f(n) = \sqrt{n}$

Step 4: Calculate

$$\log_2(2)$$

Step 5: Compare $\sqrt{n}$ with $n^{(\log_2 2)}$.

Step 6: Apply the suitable Master Theorem case.

Step 7: Display the time complexity.

Step 8: Stop the program.

Source Code

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main()
5  {
6      int n;
7      int a = 2, b = 2;
8
9      printf("Enter audio input size (n): ");
10     scanf("%d", &n);
11
12     double logValue = log(a) / log(b);
13
14     printf("\nMaster Theorem Analysis\n");
15     printf("-----\n");
16     printf("Recurrence : T(n) = 2T(n/2) + sqrt(n)\n");
17     printf("a = %d\n", a);
18     printf("b = %d\n", b);
19     printf("f(n) = sqrt(n)\n");
20     printf("log_b(a) = %.0lf\n", logValue);
21
22     printf("\nCase 1 of Master Theorem\n");
23     printf("Time Complexity = Theta(n)\n");
24
25     printf("\nAnalysis:\n");
26     printf("The speech recognition system processes\n");
27     printf("large audio inputs with linear time complexity.\n");
28
29     return 0;
30 }
```

Input and Output

```
Enter audio input size (n): 1024

Master Theorem Analysis
-----
Recurrence : T(n) = 2T(n/2) + sqrt(n)
a = 2
b = 2
f(n) = sqrt(n)
log_b(a) = 1

Case 1 of Master Theorem
Time Complexity = Theta(n)

Analysis:
The speech recognition system processes
large audio inputs with linear time complexity.

...Program finished with exit code 0
Press ENTER to exit console.
```

Performance Analysis

The algorithm divides the audio input into **2 equal parts** and processes each part recursively.

The extra work at each recursive level is only $\sqrt{n}$, which is smaller than **n**.

As the audio input size increases, the processing time increases linearly.

Therefore, the overall time complexity is:

$\Theta(n)$

This makes the algorithm efficient for processing large audio inputs because the running time grows proportionally with the input size.

Applications

- Speech recognition systems
- Voice assistants
- Audio signal processing
- Speech-to-text systems
- Voice-controlled devices
- Artificial Intelligence applications

Advantages

- Efficient for large audio inputs.
- Linear time complexity.
- Easy to analyze using the Master Theorem.
- Suitable for divide-and-conquer algorithms.
- Scalable for real-time speech processing.

Limitations

- Applicable only to recurrence relations.
- Assumes divide-and-conquer strategy.
- Cannot analyze every recursive algorithm.

Result

The recurrence relation

$$T(n) = 2T(n/2) + \sqrt{n}$$

was analyzed using the Master Theorem.

The parameters are:

- $a = 2$
- $b = 2$
- $f(n) = \sqrt{n}$

Since

$$\sqrt{n} = O(n^{1-\epsilon})$$

the recurrence satisfies **Case 1** of the Master Theorem.

Therefore, the time complexity is:

$$\Theta(n)$$

Thus, the speech recognition processing algorithm is efficient and scales linearly with larger audio input sizes.